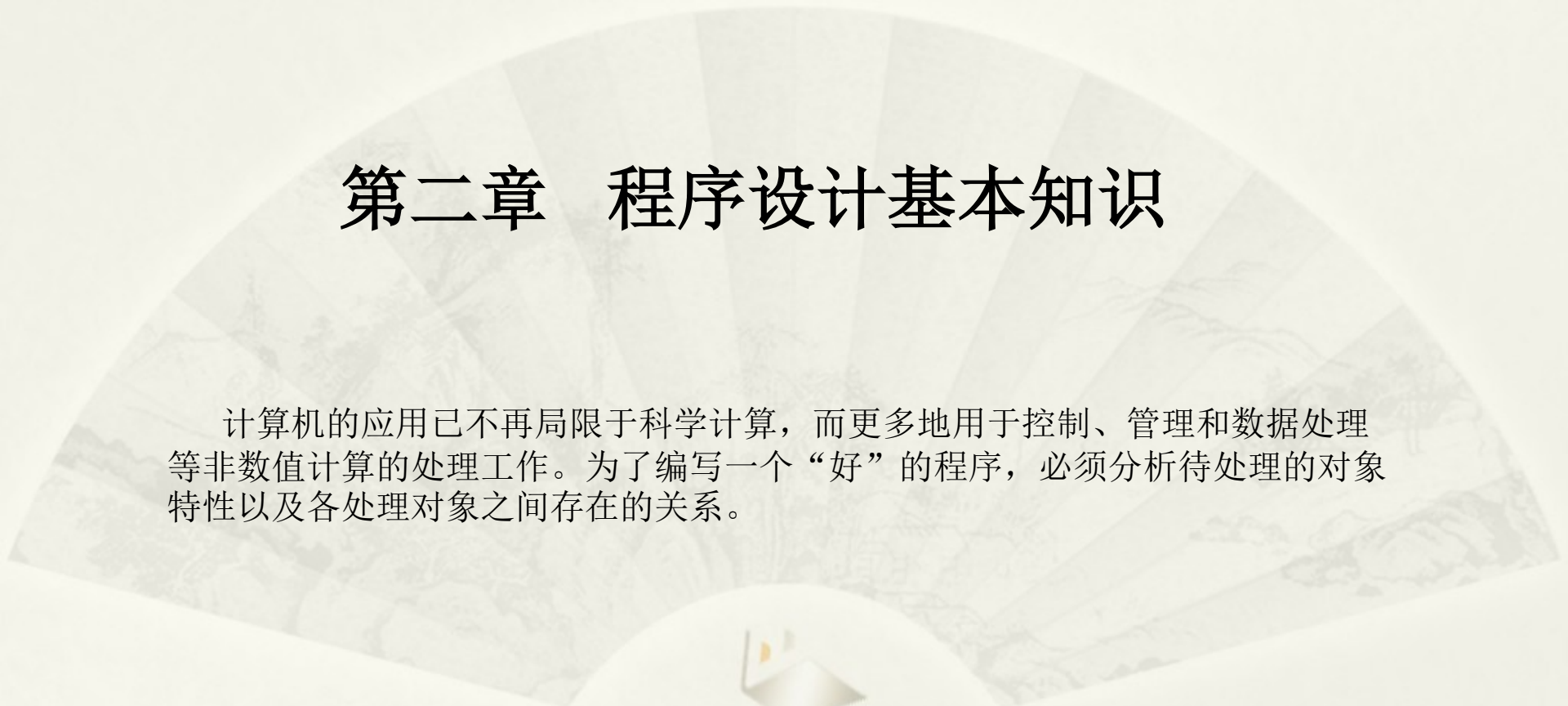




第二章 程序设计基本知识

计算机的应用已不再局限于科学计算，而更多地用于控制、管理和数据处理等非数值计算的处理工作。为了编写一个“好”的程序，必须分析待处理的对象特性以及各处理对象之间存在的关系。





第1节 程序基本常识





第1节 程序基本常识

1. 算法

算法是对特定问题求解步骤的一种描述，它是指令（规则）的有限序列，其中每一条指令表示一个或多个操作。简单地说，算法就是解决问题的操作步骤。一个算法必须满足以下五个重要的特征。

①有穷性

对于任意一组合法的输入值，算法的操作每个操作步骤都能在有限的时间内完成。这包括合理的执行时间的含义，如果一个算法执行耗费的时间太长，即使最终得出了结果，也是没有意义的。

②确定性

算法中的每一步都必须是有明确的定义，不允许有歧义性和多义性。确定性使算法的执行者或者阅读者能够明确其含义及如何执行，并且在任何条件下，算法都只有一条执行路径。

③输入

一个算法应该有0个或多个输入，以刻画运算对象的初始情况。所谓0个输入是指有的算法表面上可以没有输入，实际上已被嵌入算法之中。

④输出

一个算法应该有一个或多个输出，以反映对输入数据加工后的结果。没有输出的算法是毫无意义的；

⑤可行性

一个算法必须遵循特定条件下的解题规则，算法描述的每一个操作都应该是特定的解题规则中允许使用的、可执行的，并可以通过执行有限次来实现。



第1节 程序基本常识

2. 算法的复杂度

同一个问题可用不同的算法来解决，而算法质量的优劣将影响到算法乃至程序的效率。算法分析的目的在于选择合适的算法和改进算法。算法评价主要从时间复杂度和空间复杂度来考虑。

①时间复杂度

算法的时间复杂度（Time Complexity）是指算法所需要的计算工作量，用算法所执行的基本运算次数来度量。常见的时间复杂度有：常数阶 $O(1)$ ，对数阶 $O(\log_2 n)$ ，线性阶 $O(n)$ ，线性对数阶 $O(n \log_2 n)$ ，平方阶 $O(n^2)$ ，立方阶 $O(n^3)$ ，……，指数阶 $O(2^n)$ 。随着问题规模 n 的不断增大，上述时间复杂度不断增大，算法的执行效率就越低。

②空间复杂度

算法的空间复杂度（Space Complexity）是指执行这个算法所需要的内存空间。算法执行期间所需的存储空间主要包括三部分：输入数据所占的存储空间、程序本身所占的空间和算法执行过程中时所需的存储空间。

3. 算法的基本结构

算法的结构不仅决定了算法中各操作的执行顺序，而且也直接反映了算法的设计是否符合结构化原则。一般一个算法可以用顺序、选择和循环三种基本结构组合而成。

①顺序结构。

顺序结构是最简单、最常用的算法结构，语句与语句之间，框与框之间按从上到下的顺序进行。

②选择结构。

选择结构是先根据条件做出判断，再决定执行哪一种操作的算法结构，它必须包含判断框。当条件 P 成立（或称为真）时执行 A ，否则执行 B ，不可能两者同时执行，但 A 或 B 两个框中可以有一个是空的，即不执行任何操作。

③循环结构。

一些算法中经常出现从某处开始，按照一定条件，反复执行某一处理步骤的情况，这就是循环结构。反复执行的处理步骤为循环体，它可以细分为两类：当型循环结构和直到型循环结构。



第1节 程序基本常识

4. 基础算法

①高精度计算

利用计算机进行数值计算，有时会遇到这样的问题：有些计算要求精度高，希望计算的数的位数可达几十位甚至几百位，虽然计算机的计算精度不断提高了，但因受到硬件的限制，往往达不到实际问题所要求的精度。我们可以利用程序设计的方法去实现这样的高精度计算。主要涉及高精度的加、减、乘、除运算。实现的时候采用逐位加、减、乘、除即可。

常见问题有“汉诺塔的步数”、“国王的米粒”等。

②穷举算法

所谓穷举法，即枚举法，指的是从可能的解的集合中一一枚举各元素，用题目给定的检验条件判定哪些是无用的，哪些是有用的。能使命题成立，即为其解。

有些问题可以用循环语句和条件语句直接求解，有些问题用循环求解时循环次数太多，无法编写程序，则需要用到回溯、递归、分治等方法。

常见问题有“百钱买百鸡”、“素数判断”等。

③递推算法

递推法是一种重要的数学方法，在数学的各个领域中都有广泛的运用，也是计算机用于数值计算的一个重要算法。这种算法特点是：一个问题的求解需一系列的计算，在已知条件和所求问题之间总存在着某种相互联系的关系，在计算时，如果可以找到前后过程之间的数量关系（即递推式），那么，从问题出发逐步推到已知条件，此种方法叫逆推。无论顺推还是逆推，其关键是要找到递推式。这种处理问题的方法能使复杂运算化为若干步重复的简单运算，充分发挥出计算机擅长于重复处理的特点。递推算法的首要问题是得到相邻的数据项间的关系（即递推关系）。递推算法避开了求通项公式的麻烦，把一个复杂的问题的求解，分解成了连续的若干步简单运算。一般说来，可以将递推算法看成是一种特殊的迭代算法。

常见问题有“斐波那契数列”、“过河卒”等。



第1节 程序基本常识

④数据排序

排序就是将杂乱无章的数据元素，通过一定的方法按关键字顺序排列的过程。

排序的方法很多，下面根据初赛的要求，简单介绍各种常见排序算法在一些方面的比较，尤其是时间复杂度和稳定性两个方面。稳定性，指在原序列中相同元素的相对位置与排好序的新序列中相同元素的相对位置是否相同。若相同，则该算法是稳定的，否则不稳定。

时间复杂性比较

插入排序、冒泡排序、选择排序的时间复杂性为 $O(n^2)$ ；

其它非线形排序的时间复杂性为 $O(n\log n)$ ；

线形排序的时间复杂性为 $O(n)$ 。

稳定性比较

插入排序、冒泡排序、二叉树排序、归并排序及其他线形排序是稳定的；

选择排序、希尔排序、快速排序、堆排序是不稳定的；

辅助空间的比较

线形排序、归并排序的辅助空间为 $O(n)$ ，其它排序的辅助空间为 $O(1)$ 。

其它比较

(1) 插入、冒泡排序的速度较慢，但参加排序的序列局部或整体有序时，这种排序能达到较快的速度。反而在这种情况下，快速排序慢了，时间复杂度会达到其上限。当数据为随机数据时，快速排序远远快于插入、冒泡、选择排序，时间复杂度接近其下限。

(2) 当 n 较小时，对稳定性不作要求时宜用选择排序，对稳定性有要求时宜用插入或冒泡排序。

(3) 若待排序的记录的关键字在一个明显有限范围内时，且空间允许是用桶排序。

(4) 当 n 较大时，关键字元素比较随机，对稳定性没要求宜用快速排序。

(5) 当 n 较大时，关键字元素可能出现本身是有序的，对稳定性有要求时，空间允许的情况下，宜用归并排序。

常见问题有“合并果子”、“逆序对问题”等。



第1节 程序基本常识

⑤递归算法

递归程序设计是C++语言程序设计中的一种重要的方法，它使许多复杂的问题变得简单，容易解决了。递归特点是：函数或过程调用它自己本身。其中直接调用自己称为直接递归，而将A调用B，B以调用A的递归叫做间接递归。

常见问题有“汉诺塔问题”、“Ackermann函数”等。

⑥搜索与回溯算法

搜索与回溯是计算机解题中常用的算法，很多问题无法根据某种确定的计算法则来求解，可以利用搜索与回溯的技术求解。回溯是搜索算法中的一种控制策略。它的基本思想是：为了求得问题的解，先选择某一种可能情况向前探索，在探索过程中，一旦发现原来的选择是错误的，就退回一步重新选择，继续向前探索，如此反复进行，直至得到解或证明无解。

如迷宫问题：进入迷宫后，先随意选择一个前进方向，一步步向前试探前进，如果碰到死胡同，说明前进方向已无路可走，这时，首先看其它方向是否还有路可走，如果有路可走，则沿该方向再向前试探；如果已无路可走，则返回一步，再看其它方向是否还有路可走；如果有路可走，则沿该方向再向前试探。按此原则不断搜索回溯再搜索，直到找到新的出路或从原路返回入口处无解为止。

常见问题有“八皇后问题”、“骑士游历问题”等。

⑦贪心算法

贪心算法是指，在对问题求解时，总是做出在当前看来是最好的选择。也就是说，不从整体最优上加以考虑，他所做出的仅是在某种意义上的局部最优解。

贪心算法没有固定的算法框架，算法设计的关键是贪心策略的选择。必须注意的是，贪心算法不是对所有问题都能得到整体最优解，选择的贪心策略必须具备无后效性，即某个状态以后的过程不会影响以前的状态，只与当前状态有关。所以对所采用的贪心策略一定要仔细分析其是否满足无后效性。

常见问题有“删数问题”、“排队打水问题”等。



第1节 程序基本常识

⑧分治算法

分治就是指的分而治之，即将较大规模的问题分解成几个较小规模的问题，通过对较小规模问题的求解达到对整个问题的求解。当我们将问题分解成两个较小问题求解时的分治方法称之为二分法。

常见问题有“归并排序”、“比赛日程安排”等。

⑨广度优先搜索

广度优先算法的核心思想是：从初始节点开始，应用算符生成第一层节点，检查目标节点是否在这些后继节点中，若没有，再用产生式规则将所有第一层的节点逐一扩展，得到第二层节点，并逐一检查第二层节点中是否包含目标节点。若没有，再用算符逐一扩展第二层的所有节点……，如此依次扩展，检查下去，直到发现目标节点为止。这种搜索的次序体现沿层次向横向扩展的趋势，所以称之为广度优先搜索。

常见问题有“分油问题”、“八数码问题”等。

⑩动态规划

动态规划程序设计是解决多阶段决策过程最优化问题的一种途径、一种方法，而不是一种特殊算法。由于各种问题的性质不同，确定最优解的条件也互不相同，因而动态规划的设计方法对不同的问题，有各具特色的解题方法，而不存在一种万能的动态规划算法，可以解决各类最优化问题。需要满足“最优子结构”和“无后效性”的两项基本条件。实现时主要分成几个步骤：划分阶段、确定状态和状态变量、确定决策并写出状态转移方程、寻找边界条件、程序设计实现。大致可以分为以下几类：线性、区间、背包型、树型等。

常见问题有“最长不下降序列”、“最长公共子序列”等。



第1节 程序基本常识

【课堂练习】

1、【NOIP1998提高组】表达式 $(4 \% (-3))$ 与 $(-4 \% 3)$ 的值为 ()。

A. -1, -1 B. 1, -1 C. -1, 1 D. 1, 1

【答案】B

【分析】任何一个整数 n 都可以表示成 $n=k*q+r$ 其中 $0 \leq |r| < |q|$, r 就是 n 除以 q 的余数, 即 $r=n \% q$ 。

例如: $-9=(-2)*4+(-1)$ 则 -9 除以 4 的余数为 -1 。就是说被除数是负数, 余数是负数, 被除数是正数, 余数是正数。

2、【NOIP2000】已知数组 A 中, 每个元素 $A[I][J]$ 在存贮时要占3个字节, 设 I 从1变化到8, J 从1变化到10, 分配内存时是从地址 SA 开始连续按行存贮分配的。

试问: $A[5][8]$ 的起始地址为 ()。

A. $SA+141$ B. $SA+180$ C. $SA+222$ D. $SA+225$

【答案】A

【分析】设数组有 m 行 n 列, 可以总结出公式: $A[I][J]$ 的起始地址 = $SA + ((I-1)*n + ((J-1))*3)$, 将 $m=8$, $n=10$ 代入以上公式得 $SA + ((5-1)*10 + ((8-1))*3) = SA + 141$ 。

3、【NOIP2002】一个向量第一个元素的存储地址是100, 每个元素的长度是2, 则第5个元素的地址是 ()。

A. 110 B. 108 C. 100 D. 109

【答案】B

【分析】数据元素的存储位置均取决于第1个数据元素的存储位置, 即 $LOC(a_i) = LOC(a_1) + (i-1) \times C$, 所以第5个元素的地址为 $100 + 2 * (5-1) = 108$ 。

4、【NOIP2006】在编程时(使用任一种高级语言, 不一定是 C++), 如果需要从磁盘文件中输入一个很大的二维数组(例如 $1000*1000$ 的 `double` 型数组), 按行读(即外层循环是关于行的)与按列读(即外层循环是关于列的)相比, 在输入效率上 ()。

A. 没有区别 B. 按行读的方式要高一些
C. 按列读的方式要高一些 D. 取决于数组的存储方式。

【答案】D

【分析】按行, 按列, 决定元素是否连续, 可能会有差异。



第1节 程序基本常识

5、【NOIP2000普及组】算法是指（ ）。

- A. 为解决问题而编制的计算机程序
- B. 为解决问题而采取的方法与步骤
- C. 为解决问题而需要采用的计算机语言
- D. 为解决问题而采用的计算方法

【答案】B

【分析】算法是对特定问题求解步骤的一种描述，它是指令（规则）的有限序列，其中每一条指令表示一个或多个操作。简单地说，算法就是解决问题的操作步骤

6、【NOIP2001】下面关于算法的错误说法是（ ）。

- A. 算法必须有输出
- B. 算法必须在计算机上用某种语言实现
- C. 算法不一定有输入
- D. 算法必须在有限步执行后能结束

【答案】B

【分析】算法除了以计算机语言实现外，还可以框图、文字等方式存在。

7、【NOIP2006普及组】在下列关于计算机算法的说法中，不正确的是（ ）。

- A. 一个正确的算法至少要有有一个输入
- B. 算法的改进，在很大程度上推动了计算机科学与技术的进步
- C. 判断一个算法的好坏的主要标准是算法的时间复杂性与空间复杂性
- D. 目前仍然存在许多涉及到国计民生的重大课题，还没有找到能够在计算机上实施的有效算法

【答案】A

【分析】算法（algorithm）是在有限步骤内求解某一问题所使用的一组定义明确的规则。

一个算法应该具有以下五个重要的特征：

有穷性：一个算法必须保证执行有限步之后结束；

确切性：算法的每一步必须有确切的定义；

输入：一个算法有0个或多个输入，以刻画运算对象的初始情况，所谓0个输入是指算法本身定出了初始条件；

输出：一个算法有一个或多个输出，以反映对输入数据加工后的结果。没有输出的算法是毫无意义的；

可行性：算法原则上能够精确地运行，而且人们用笔和纸做有限次运算后即可完成；

8、【NOIP2000普及组】在待排序的数据表已经为有序时，下列排序算法中花费时间反而多的是（ ）。

- A. 堆排序
- B. 希尔排序
- C. 冒泡排序
- D. 快速排序

【答案】D

【分析】最坏情况下，是整个序列都已经有序，此时，快速排序退化为冒泡排序，要比较 $n*(n-1)/2$ 次才能完成。



第1节 程序基本常识

9、【NOIP2002】在所有排序方法中,关键字比较的次数与记录的初始排列次序无关的是()。

A. 希尔排序 B. 起泡排序 C. 插入排序 D. 选择排序

【答案】D

【分析】选择排序特点是排序总是从第一位开始,与起始位无关。所有趟数下来关键字总是总共比较 $n(n-1)/2$ 次,每一趟关键字比较次数也是固定的。

10、【NOIP2006普及组】在下列各种排序算法中,不是以“比较”作为主要操作的算法是()。

A. 选择排序 B. 冒泡排序 C. 插入排序 D. 基数排序

【答案】D

【分析】基数排序按个、十、百……轮流排序,非比较。

11、【NOIP2006】将5个数的序列排序,不论原先的顺序如何,最少都可以通过()次比较,完成从小到大的排序。

A. 6 B. 7 C. 8 D. 9

【答案】B

【分析】简单的说,5个数的全排列,共有 $5*4*3*2*1=120$ 种,而每次比较能得到或大或小两种情况, n 次比较可得到 2^n 种情况,所以要想区分出这120种情况,至少要 $n=7$,即有 $2^7=128 \geq 120$ 。也就是说至少要7次比较。

或者:

5个数7次排好,具体如下:

假设5个数分别编号1, 2, 3, 4, 5,

1v2, 3v4.....2次,假设1>2, 3>4;

1v3,1次,假设1>3则1>3>4;

把5插入1, 3, 4中,需要2次,

结果有4种:

5>1>3>4, 则把2插入3, 4中,需2次

1>5>3>4, 则把2插入5, 3, 4中,需2次

1>3>5>4, 同上

1>3>4>5, 同上

这样一共 $2+1+2+2=7$ 。





第1节 程序基本常识

12、【NOIP2000】某数列有1000个各不相同的单元，由低至高按序排列；现要对该数列进行二分法检索(binary search)，在最坏的情况下，需检视()个单元。

- A. 1000 B. 10 C. 100 D. 500

【答案】B

【分析】由二分法定义得： $2^9 < 1000 < 2^{10}$ ，需检视10个单元。

13、【NOIP2001】在顺序表(2, 5, 7, 10, 14, 15, 18, 23, 35, 41, 52)中，用二分法查找12，所需的关
键码比较的次数为()。

- A. 2 B. 3 C. 4 D. 5

【答案】C

【分析】按二分法，该数需依次和15, 7, 10, 14比较，共4次。

14、【NOIP2004提高组】由3个a, 5个b和2个c构成的所有字符串中，包含子串“abc”的共有()个。

- A. 40320 B. 39600 C. 840 D. 780 E. 60

【答案】D

【分析】排列组合。

方法一：将abc当做d，原问题可以分为{2a,4b,1c,1d}和{1a,3b,2d}

{1a,3b,2d}— $6! / (3! * 2!) = 60$ 3!是b的重复，2!为d的重复

{2a,4b,1c,1d}— $8! / (2! * 4!) = 840$

$840 - 60 = 780$

方法二：2个a，4个b，1个c，1个d(abc)的可重排列，

$(2+4+1+1)! / (2! * 4!) = 840$ 再减去1个a，3个b，2个d(abc)的多重排列， $6! / (2! * 3!) = 60$ ，60为有
出现两个abc子串的情况， $840 - 60 = 780$

方法三： $(8 * 7!) / (2! * 4!) - (5+4+3+2+1) * 4 = 780$

亦即，考虑“abc”的摆放位置，共有8种，余下的7个字符的全排列有7!种。但是，在这些7!种全排列中，a的重复摆放共有2!种，b的重复摆放有4!种。此外，在余下的7个字符中，仍有可能出现“abc”的排列，这与前面考虑的8种“abc”的摆放是重复的，也要去掉。这时，根据头一个“abc”的摆放起点位置，后一个“abc”分别有5,4,3,2,1种可能的摆放位置，而一旦第二个“abc”摆放好后，余下的一个a和三个b的摆放位置C(4,1)种可能，因而得上式。



第1节 程序基本常识

17、【NOIP2007】在关系数据库中，存放在数据库中的数据的逻辑结构以（ ）为主。

- A. 二叉树 B. 多叉树 C. 哈希表 D. 二维表

【答案】D

【分析】在关系数据库中，存在数据库中的数据的逻辑结构以二维表为主，数据库中的表横行为记录，列标题为字段，类似excel表格，但又不是电子表格。

18、【NOIP2000】线性表若采用链表存贮结构，要求内存中可用存贮单元地址（ ）。

- A. 必须连续 B. 部分地址必须连续
C. 一定不连续 D. 连续不连续均可

【答案】D

【分析】如果用链表，每个元素都有一个指针域，指向下一个节点，所以可以不连续，当然连续也可以。如果换成数组，那么必须连续，因为数组本身就是连续的。

19、【NOIP2000】下列叙述中，正确的是（ ）。

- A. 线性表的线性存贮结构优于链表存贮结构
B. 队列的操作方式是先进后出
C. 栈的操作方式是先进先出
D. 二维数组是指它的每个数据元素为一个线性表的线性表

【答案】D

【分析】链表存贮结构在某些方面线性存贮结构，队列的操作方式是先进先出，堆栈的操作方式是后进先出，因式A、B、C都是错误的。



第1节 程序基本常识

15、【NOIP2005提高组】字符串“ababacbab”和字符串“abcba”的最长公共子串是()。

A. abcba B. cba C. abc D. ab E. bcba

【答案】B

【分析】暴力模拟一下，或者动态规划一下。

16、【NOIP2000】电线上停着两种鸟(A, B)，可以看出两只相邻的鸟就将电线分为一个线段。这些线段可分为两类：一类是两端的小鸟相同；另一类则是两端的小鸟不相同。已知：电线两个顶点上正好停着相同的小鸟，试问两端为不同小鸟的线段数目一定是()。

A. 奇数 B. 偶数 C. 可奇可偶 D. 数目固定

【答案】B

【分析】本题可考虑为每增加一只小鸟，两端为不同小鸟的线段数目的奇偶性是否发生变化。设想第一种情况新增加的小鸟(如A)落在了同类鸟(如AA)中间，则两端为不同小鸟的线段数目不变，奇偶性不发生变化；第二种情况新增加的小鸟(如A)落在了两只同类鸟(如BB)中间，两端为不同小鸟的线段数目增加2，奇偶性不发生变化；第三种情况新增加的小鸟(如A)落在了一只同类鸟和一只不同类鸟(AB或BA)，则两端为不同小鸟的线段数目减少一个，增加一个，总数不变，奇偶性也不发生变化。可设定开始只有两只相同的小鸟停在两端两端为不同小鸟的线段数目为0，是偶数，小鸟一只只飞进来，都不改变其奇偶性，因此一定是偶数。



第2节 逻辑运算





第2节 逻辑运算

例题1. 若 $A=True$, $B=False$, $C=True$, $D=False$, 以下逻辑运算表达式真的有 ()。

- A. $(A \wedge B) \vee (C \wedge D \vee \neg A)$ B. $((\neg A \wedge B) \vee C) \wedge \neg B$
C. $(B \vee C \vee D) \vee D \wedge A$ D. $A \wedge (D \vee \neg C) \wedge B$

题解: 一个个算结果, 比如A选项 $(A \wedge B) \vee (C \wedge D \vee \neg A)$, 根据运算级的比较, 我们可以定下运算的顺序, 然后按运算顺序计算结果。注意, 这类题是有个小技巧的。比如A选项可以先看中间的 $\vee$, 为什么呢? 因为 $\vee$ 的左右有一边是真就行, 可以不去看另外一边。

A选项的结果是: $(A \wedge B) \vee (C \wedge D \vee \neg A)$, $(A \wedge B)=假$, $(C \wedge D \vee \neg A)$ 中 $C \wedge D=假$, $\neg A=假$, 所以 $(C \wedge D \vee \neg A)=假$ 。于是A选项可以简写为: 假 $\vee$ (假 $\vee$ 假)=假。

B选项的结果是: $((\neg A \wedge B) \vee C) \wedge \neg B$, 如果 $\neg B$ 是假那么就可以不去看前面的 $((\neg A \wedge B) \vee C)$, 可惜的是 $\neg B$ 是真, 那么就要看 $((\neg A \wedge B) \vee C)$, 发现C是真, 所以不看 $(\neg A \wedge B)$, 于是B选项可以简写为: (? $\vee$ 真) $\wedge$ 真 = 真。

C选项的结果是: $(B \vee C \vee D) \vee D \wedge A$, $D \wedge A=假$, 所以不得不看前面部分 $(B \vee C \vee D)$, 只要BCD有一个是真, 那么 $(B \vee C \vee D)=真$, 而容易发现 $C=true$ 。所以C选项可以简写为: 真 $\vee$ 假 = 真。

D选项的结果是: $A \wedge (D \vee \neg C) \wedge B$, 我们很容易发现D选项的特殊结构为 ? $\wedge$? $\wedge$? , 三个? 有一个是假, 那么D为假, A和B不用计算便可看出, 所以先发现 $B=假$, 所以 $D=假$ 。



第2节 逻辑运算

逻辑运算先掌握各种运算，注意运算符的级别比较，做题是要细心。

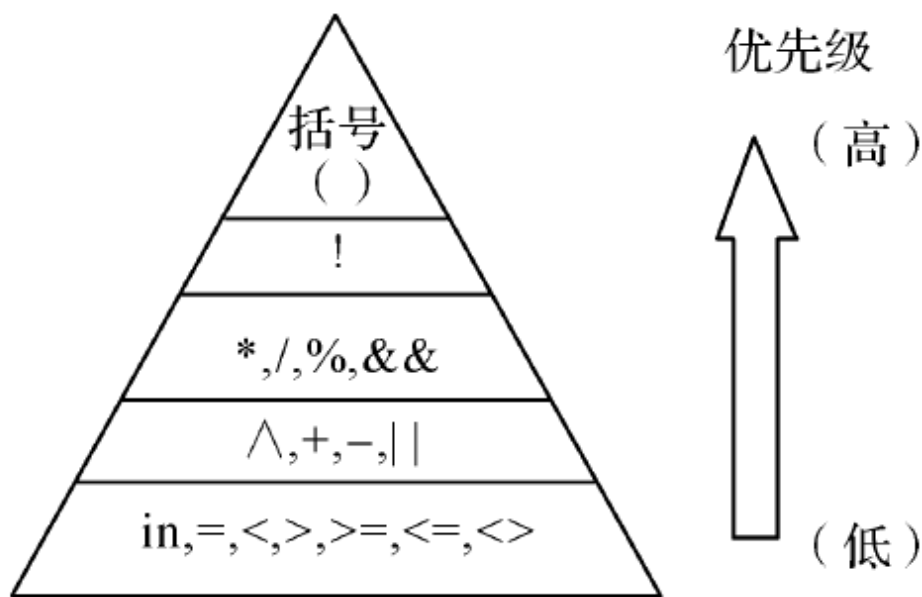
1.概念介绍

非：！ 与：&& 或：|| 异或：^

2.运算级比较

括号 > 非 > 与 > 或、异或 （||和^是同级的）

如果加入加减乘除，就是以下这样：



注意：同级的运算符不分高低，计算时按照从左到右运算。



第2节 逻辑运算

【课堂练习】

1、【NOIP2002提高组】已知 $A = 35H$ ， $A \wedge 05H \vee A \wedge 30H$ 的结果是（ ）。

A. 30H B. 05H C. 35H D. 53H

【答案】C

【分析】将上述十六进制数转化成二进制岁数，逐位进行逻辑运算，注意与运算比或运算的优先级高。

$35H = 0011\ 0101\ B$ $05H = 0000\ 0101\ B$ $30H = 0011\ 0000\ B$

$A \wedge 05H = 0011\ 0101B \wedge 0000\ 0101B = 0000\ 0101B$

$A \wedge 30H = 0011\ 0101B \wedge 0011\ 0000B = 0011\ 0000B$

$A \wedge 05H \vee A \wedge 30H = 0000\ 0101B \vee 0011\ 0000B = 0011\ 0101B = 35H$

2、【NOIP2003】假设 $A = \text{true}$ ， $B = \text{false}$ ， $C = \text{true}$ ， $D = \text{true}$ ，逻辑运算表达式 $A \wedge B \vee C \wedge D$ 的值是（ ）。

A. true B. false C. 0 D. 1 E. NULL

【答案】A

【分析】注意“与”运算比“或”运算的优先级高，所以 $A \wedge B$ 和 $C \wedge D$ 先运算，分别得到false和true，然后 $\text{false} \vee \text{true}$ 得到true。

3、【NOIP2003提高组】设全集 $E = \{1, 2, 3, 4, 5\}$ ，集合 $A = \{1, 4\}$ ， $B = \{1, 2, 5\}$ ， $C = \{2, 4\}$ ，则集合 $(A \cap B) \cup \sim C$ 为（ ）。

A. 空集 B. $\{1\}$ C. $\{3, 5\}$ D. $\{1, 5\}$ E. $\{1, 3, 5\}$

【答案】E

【分析】集合运算的一元操作符优先级高于二元，求“补”的优先级比“交”和“并”高，交运算和并运算优先级相同。 $A \cap B$ 是交运算，结果是 $\{1\}$ ， $\sim C$ 的结果是 $\{1, 3, 5\}$ ，最后进行的运算是 $\{1\} \cup \{1, 3, 5\}$ ，最终结果为 $\{1, 3, 5\}$ 。

4、【NOIP2004提高组】设全集 $I = \{a, b, c, d, e, f, g\}$ ，集合 $A = \{a, b, c\}$ ， $B = \{b, d, e\}$ ， $C = \{e, f, g\}$ ，那么集合为（ ）。

A. $\{a, b, c, d\}$ B. $\{a, b, d, e\}$ C. $\{b, d, e\}$ D. $\{b, c, d, e\}$ E. $\{d, f, g\}$

【答案】A

【分析】考查集合知识。



第2节 逻辑运算

例题2. 计算 $23 + 2 || 2 \&\& 5 * 3 - 6 ^ 5 = ()$ 。

题解：数字也有逻辑运算，当然也可以混合加减乘除。

这里举例说明运算的操作：

&&: 22 && 5		: 22 or 5	
22: 10110	→ 10110	22: 10110	→ 10110
5: <u> 101 </u> (缺位补零)	<u>00101</u>	5: <u> 101 </u> (缺位补零)	<u>00101</u>
(垂直对应两位 && 运算)	00100 = 4	(垂直对应两位 运算)	00100 = 4

2004和2005都出现了**集合运算**，虽然后来没有再出现，但集合的运算还是需要掌握的。

并运算：； **交运算：**； **差运算：**； **非运算：**（区别于逻辑非运算：）

并运算：比如AB，就是A集合和B集合里所有元素组成一个新集合，重复的元素只保留一份。 $AB \rightarrow \{a, b, c, b, d, e\} \rightarrow \{a, b, c, d, e\}$ 。

交运算：比如AB，就是同时在A集合和B集合的元素组成一个新集合。 $AB \rightarrow \{b\}$ 。

差运算：比如AB，就是A集合删去AB里的元素后组成一个新集合。 $AB \rightarrow \{a, c\}$ 。

非运算：如是单目运算符，比如A。非运算有个特殊的要求：一定要说明全集。那么A就全集删去A集合中的元素，剩下的全集中的元素组成一个新集合。 $A = \{d, e, f, g\}$ 。



第2节 逻辑运算

5、【NOIP2005普及组】设 $A = \text{true}$, $B = \text{false}$, $C = \text{false}$, $D = \text{true}$, 以下逻辑运算表达式值为真的是 ()。

- A. $(A \wedge B) \vee (C \wedge D)$ B. $((A \wedge B) \vee C) \wedge D$ C. $A \wedge ((B \vee C) \wedge D)$
D. $(A \wedge (B \vee C)) \vee D$ E. $(A \vee B) \wedge (C \wedge D)$

【答案】D

【分析】考查逻辑运算的知识, $\wedge$ 相当于与 $\&\&$, $\vee$ 相当于或 $\|\|$ 。

6、【NOIP2005提高组】设全集 $I = \{a, b, c, d, e, f, g, h\}$, 集合 $B \subseteq A = \{a, b, c, d, e, f\}$, $C \subseteq A = \{c, d, e\}$, $\sim B \cap A = \{a, d\}$, 那么集合 $C \cap B \cap A$ 为 ()。

- A. $\{c, e\}$ B. $\{d, e\}$ C. $\{e\}$ D. $\{c, d, e\}$ E. $\{d, f\}$

【答案】A

【分析】考查集合知识。

7、【NOIP2006普及组】设 $A=B=D=\text{true}$, $C=\text{false}$, 以下逻辑运算表达式值为真的有 ()。

- A. $(\neg A \wedge B) \vee (C \wedge D)$ B. $\neg ((A \vee B \vee D) \wedge C)$
C. $\neg A \wedge (B \vee C \vee D)$ D. $(A \wedge B \wedge C) \vee \neg D$

【答案】B

【分析】 $\wedge$ 相当于与 $\&\&$, $\vee$ 相当于或 $\|\|$, $\neg$ 相当于非。

8、【NOIP2007普及组】设 $A=B=\text{true}$, $C=D=\text{false}$, 以下逻辑运算表达式值为假的有 ()。

- A. $(\neg A \wedge B) \vee (C \wedge D \vee A)$ B. $\neg (((A \wedge B) \vee C) \wedge D)$
C. $A \wedge (B \vee C \vee D) \vee D$ D. $(A \wedge (D \vee C)) \wedge B$

【答案】D

【分析】考查逻辑运算的知识, $\wedge$ 相当于与 $\&\&$, $\vee$ 相当于或 $\|\|$, $\neg$ 相当于非。



第2节 逻辑运算

9、【NOIP2006普及组】在C++中，表达式 $21 \wedge 2$ 的值是（ ）。

- A. 441 B. 42 C. 23 D. 24

【答案】C

【分析】 $\wedge$ 异或操作。

21转换为二进制10101, 2转换为二进制00010, 两数异或得到结果10111, 再将其转换为十进制数, 得到结果23。

	16	8	4	2	1
	1	0	1	0	1
xor	0	0	0	1	0
	1	0	1	1	1

10、【NOIP2006】在C++中，判断a不等于0且b不等于0的正确的条件表达式是（ ）。

- A. $!a == 0 \parallel !b == 0$ B. $!((a == 0) \&\& (b == 0))$ C. $!(a == 0 \&\& b == 0)$ D. $a \&\& b$

【答案】D

【分析】0为假，非0为真。

11、【NOIP2007】在C++程序中，表达式 $23 | 2 \wedge 5$ 的值是（ ）。

- A. 23 B. 1 C. 32 D. 18

【答案】A

【分析】 $\wedge$ 为位运算异或，不同为1，相同为0， $23 = 10111(2)$ ， $2 = 10(2)$ ， $5 = 101(2)$ ，位运算优先级 $\& > \wedge > |$ ， $2 \wedge 5 = 7$ ， $23 | 7 = 23$ 。

12、【NOIP2007】在C++程序中，判断a等于0或b等于0或c等于0的正确的条件表达式是（ ）。

- A. $!((a != 0) \parallel (b != 0) \parallel (c != 0))$ B. $!((a != 0) \&\& (b != 0) \&\& (c != 0))$
C. $!(a == 0 \&\& b == 0) \parallel (c != 0)$ D. $(a = 0) \&\& (b = 0) \&\& (c = 0)$

【答案】B

【分析】DE语法错误先排除，题目要求相当于 $a == 0 \parallel b == 0 \parallel c == 0$ ，abc只要有一个为真就成立，列个 2^3 的01真假值表，可以穷举验证一下。



第3节 栈



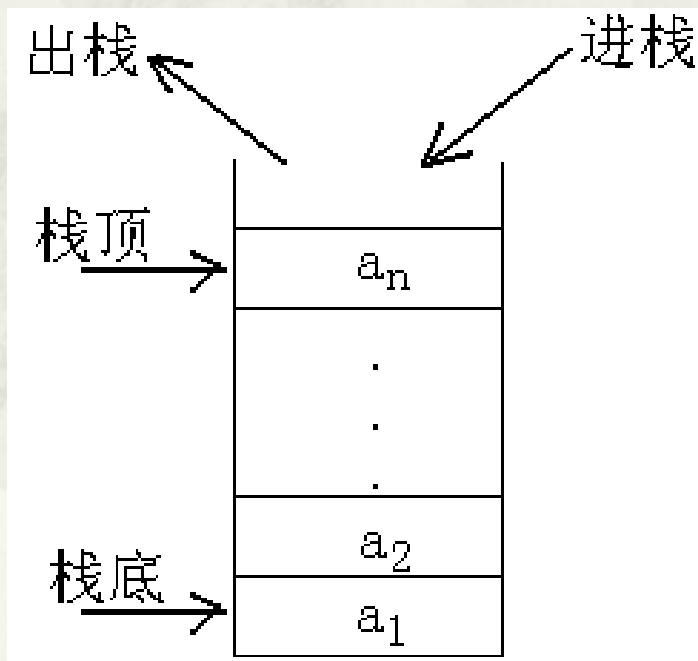


栈是只能在某一端插入和删除的特殊线性表。

用桶堆积物品，先堆进来的压在底下，随后一件一件往上堆。取走时，只能从上面一件一件取。堆和取都在顶部进行，底部一般是不动的。

栈就是一种类似桶堆积物品的数据结构，进行删除和插入的一端称栈顶，另一堆称栈底。插入一般称为进栈（PUSH），删除则称为退栈（POP）。栈也称为后进先出表（LIFO表）。

一个栈可以用定长为N的数组S来表示，用一个栈指针TOP指向栈顶。若 $TOP=0$ ，表示栈空， $TOP=N$ 时栈满。进栈时TOP加1。退栈时TOP减1。当 $TOP<0$ 时为下溢。栈指针在运算中永远指向栈顶。





1、进栈 (PUSH) 算法

①若 $TOP \geq n$ 时, 则给出溢出信息, 作出错处理 (进栈前首先检查栈是否已满, 满则溢出; 不满则作②);

② $TOP++$ (栈指针加 1, 指向进栈地址);

③ $S[TOP]=X$, 结束 (X 为新进栈的元素);

2、退栈 (POP) 算法

①若 $TOP \leq 0$, 则给出下溢信息, 作出错处理 (退栈前先检查是否已为空栈, 空则下溢; 不空则作②);

② $X=S[TOP]$, (退栈后的元素赋给 X);

③ $TOP--$, 结束 (栈指针减 1, 指向栈顶)。

进栈、出栈的c++实现过程程序:

```
#define n 100
```

```
void push(int s[],int *top,int *x) //入栈
```

```
{  
    if (*top==n) printf("overflow");  
    else { *top++; s[*top]=*x; }  
}
```

```
void pop(int s[],int *y,int *top) //出栈
```

```
{  
    if (*top==0) printf("underflow");  
    else { *y=s[*top]; *top--; }  
}
```





对于出栈运算中的“下溢”，程序中仅给出了一个标志信息，而在实际应用中，下溢可用来作为控制程序转移的判断标志，是十分有用的。对于入栈运算中的“上溢”，则是一种致命的错误，将使程序无法继续运行，所以要设法避免。

堆栈的数组模拟

十进制数N和其它d进制数的转换是实现计算的基本问题，解决方法很多，下面给出一种算法原理： $N=(N/d) \times d + N \% d$ （其中/为整除运算，%为求余运算）。

例如： $(1348)_{10}=(2504)_8$ 运算过程如下：

N	N / 8	N % 8
9413		

N	N / 8	N % 8
1348	168	4
168	21	0
21	2	5
2	0	2

填空： $(9413)_{10}=(\quad)_8=(\quad)_{16}=(\quad)_2$

数制转化程序

```
#include<cstdlib>
#include<iostream>
using namespace std;
#define size 100
int a[size+1],n,d,i=0,j;
main()
{
    cout<<"Please enter a number(N) base 10:"; cin>>n;
    cout<<"please enter a number(d):"; cin>>d;
    do{
        a[++i]=n%d;
        n=n/d;
    }while(n!=0);
    for (j=i;j>=1;j--)cout<<a[j];
    return 0;
}
```

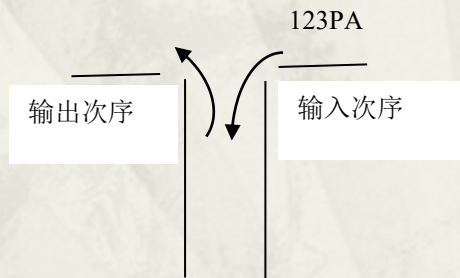


例题1. 设输入元素为1、2、3、P和A，输入次序为123PA，如图所示。元素经过栈后到达输出序列，当所有元素均到达输出序列后，有哪些序列可以作为高级语言的变量名？123PA输入次序输出次序

【解答】

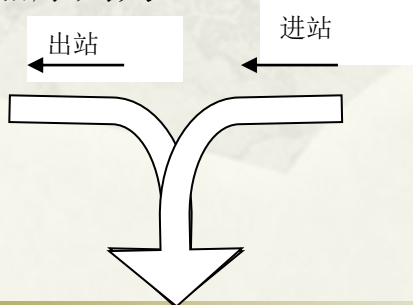
高级语言变量名的定义规则：以字母开头，字母与数字的组合。出站进站

由于必须以字母开头，所以，第一个可能出现的字母是P，那么必然要求123已经先后入栈，这样便可得到以P开头的、并且123按逆序排列的（即321）、同时A位于P后任一位置的变量名序列。此外，还需要考虑以A开头的可能情况，这只有一种情形：AP321。所以AP321，PA321，P3A21，P32A1，P321A可以作为高级语言的变量名。



例题2. 火车站列车调度示意图如下，假设调度站两侧的轨道为单向行驶轨道。

- ①如果进站的车厢序列为123，则可能的出站车厢序列是什么？
- ②如果进站的车厢序列为123456，问能否得到135426和435612的出站序列。





【课堂练习】

1、【NOIP2001】若已知一个栈的入栈顺序是1, 2, 3, ..., n, 其输出序列为P1, P2, ..., Pn, 若P1是n, 则Pi是 ()

- A. i B. n-1 C. n-i+1 D. 不确定

【答案】C

【分析】归纳总结

P1 n

P2 n - 1

P3 n - 2

.....

Pn 1

总结: Pi对应 n - i + 1

2、【NOIP2001提高组】以下哪一个不是栈的基本运算 ()。

- A. 删除栈顶元素 B. 删除栈底的元素 C. 判断栈是否为空 D. 将栈置为空栈

【答案】B

【分析】堆栈删除和插入操作只能在栈顶进行。

3、【NOIP2002提高组】设栈S和队列Q的初始状态为空, 元素e1, e2, e3, e4, e5, e6依次通过栈S, 一个元素出栈后即进入队列Q, 若出队的顺序为e2, e4, e3, e6, e5, e1, 则栈S的容量至少应该为()。

- A. 2 B. 3 C. 4 D. 5

【答案】B

【分析】因为依次入栈, 所以一个元素入队时, 比它标号小的元素要么已入队, 要么还在栈中, 对每个元素计算比它标号小的元素总数减去队列中比它标号小的元素个数得尚在栈中的元素个数, 找出最大值加1(其本身刚出栈), 就是所要结果。例中e6入队时, 队中有e2, e4, e3共3个元素, 比e6标号小的有5个元素, 5-3+1=3。

5、【NOIP2006】某个车站呈狭长形, 宽度只能容下一台车, 并且只有一个出入口。已知某时刻该车站状态为空, 从这一时刻开始的出入记录为: “进, 出, 进, 进, 进, 出, 出, 进, 进, 进, 出, 出”。假设车辆入站的顺序为1, 2, 3,, 则车辆出站的顺序为 ()。

- A. 1, 2, 3, 4, 5 B. 1, 2, 4, 5, 7 C. 1, 4, 3, 7, 6 D. 1, 4, 3, 7, 2

【答案】C

【分析】栈按先进后出顺序, 元素上面有元素就不能出栈。



- 4、【NOIP2003提高组】已知元素(8, 25, 14, 87, 51, 90, 6, 19, 20)，问这些元素以怎样的顺序进入栈，才能使出栈的顺序满足：8在51前面；90在87的后面；20在14的后面；25在6的前面；19在90的后面()。
- A. 20, 6, 8, 51, 90, 25, 14, 19, 87 B. 51, 6, 19, 20, 14, 8, 87, 90, 25
C. 19, 20, 90, 7, 6, 25, 51, 14, 87 D. 6, 25, 51, 8, 20, 19, 90, 87, 14
E. 25, 6, 8, 51, 87, 90, 19, 14, 20

【答案】D

【分析】根据堆栈先进后出的操作原则，出栈顺序和入栈顺序相反，将题中所给两数间的前后关系颠倒过来：
18在51的前面8在51的后面290在87的后面90在87的前面320在14的后面20在14的前面425在6的前面25在6的后面
519在90的后面19在90的前面

其中：A不符合1；B不符合2；C不符合1；E不符合1。符合条件的只有D。

- 6、【NOIP2006】设栈S的初始状态为空，元素a, b, c, d, e依次入栈，以下出栈序列不可能出现的有()。
- A. a, b, c, e, d B. b, c, a, e, d
C. a, e, c, b, d D. d, c, e, b, a

【答案】C

【分析】C中a和e出栈后，从栈顶到栈底有dcb，d没有出栈c和b出不来。

- 7、【NOIP2007】地面上有标号为A、B、C的3根细柱，在A柱上放有10个直径相同中间有孔的圆盘，从上到下依次编号为1, 2, 3, ……，将A柱上的部分盘子经过B柱移入C柱，也可以在B柱上暂存。如果B柱上的操作记录为：“进，进，出，进，进，出，出，进，进，出，进，出，出”。那么，在C柱上，从下到上的盘子的编号为()。
- A. 2 4 3 6 5 7 B. 2 4 1 2 5 7 C. 2 4 3 1 7 6 D. 2 4 3 6 7 5

【答案】D

【分析】本题就是一个栈，按照“进、出……”的描述，进栈、出栈的顺序是固定的，不得调整的。这样一来，进、出栈的顺序就是：1进、2进、2出、3进、4进、4出、3出、5进、6进、6出、7进、7出、5出。



第4节 队列





队列是限定在一端进行插入，另一端进行删除特殊线性表。就像排队买东西，排在前面的人买完东西后离开队伍（删除），而后来的人总是排在队伍末尾（插入）。通常把队列的删除和插入分别称为出队和入队。允许出队的一端称为队头，允许入队的一端称为队尾。所有需要进队的数据项，只能从队尾进入，队列中的数据项只能从队头离去。由于总是先入队的元素先出队(先排队的人先买完东西)，这种表也称为先进先出(FIFO)表。

队列可以用数组 $Q[m+1]$ 来存储，数组的上界 m 即是队列所容许的最大容量。在队列的运算中需设两个指针：

head: 队头指针，指向实际队头元素的前一个位置

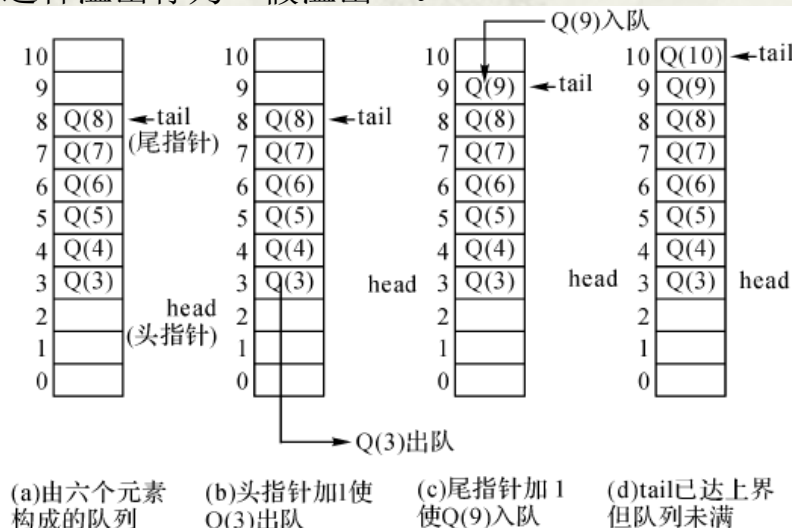
tail: 队尾指针，指向实际队尾元素所在的位置

一般情况下，两个指针的初值设为 0，这时队列为空，没有元素。图1 (a)画出了一个由 6 个元素构成的队列，数组定义 $Q[11]$ 。 $Q[i] \ i=3,4,5,6,7,8$ 头指针 $head=2$ ，尾指针 $tail=8$ 。

队列中拥有的元素个数为： $L=tail-head$ 现要让队头的元素出队，则需将头指针加 1。即 $head=head+1$ 这时头指针向上移动一个位置，指向 $Q[3]$ ，表示 $Q[3]$ 已出队。见图1 (b)。

如果想让一个新元素入队，则需尾指针向上移动一个位置。即 $tail=tail+1$ 这时 $Q[9]$ 入队，见图1 (c)。

当队尾已经处理在最上面时，即 $tail=10$ ，见图1 (d)，如果还要执行入队操作，则会发生“上溢”，但实际上队列中还有三个空位置，所以这种溢出称为“假溢出”。



【课堂练习】

1、【NOIP2000普及组】设循环队列中数组的下标范围是1~n，其头尾指针分别为f和r，则其元素个数为()。

- A. $r - f$ B. $r - f + 1$ C. $(r - f) \% n + 1$ D. $(r - f + n) \% n$

【答案】D

【分析】循环队列中f和r不一定满足f<r，如果满足f<r，则元素个数为r-f；但有时也会r<f，所以需要加n然后对n求余才能统一表示。

2、【NOIP2003】已知队列(13, 2, 11, 34, 41, 77, 5, 7, 18, 26, 15)，第一个进入队列的元素是13，则第五个出队列的元素是()。

- A. 5 B. 41 C. 77 D. 13 E. 18

【答案】B

【分析】根据队列先进先出的操作原则，第一个进入队列的是13，则第一个出队的也是13，向后依次为2, 11, 34, 41, 77, 5, 7, 18, 26, 15。其中第五个是41，因此答案为B。

3.若用一个大小为6的数组来实现循环队列，且当rear和front的值分别为0和3。当从队列中删除一个元素，再加入两个元素后，rear和front的值分别为()。

- A. 1和5 B. 2和4 C. 4和2 D. 5和1

【答案】B

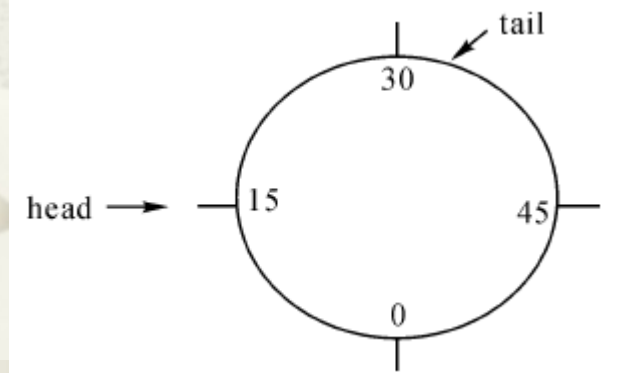
【分析】删除一个元素，则front要前进一格，即 $3+1=4$ ；加入两个元素，则rear要前进两格，即 $0+2=2$ ；所以rear和front的值分别为2和4。

4.如图所示的循环队列中元素数目是()。其中tail=32指向队尾元素，head=15指向队头元素的前一个空位置，队列空间m=60。

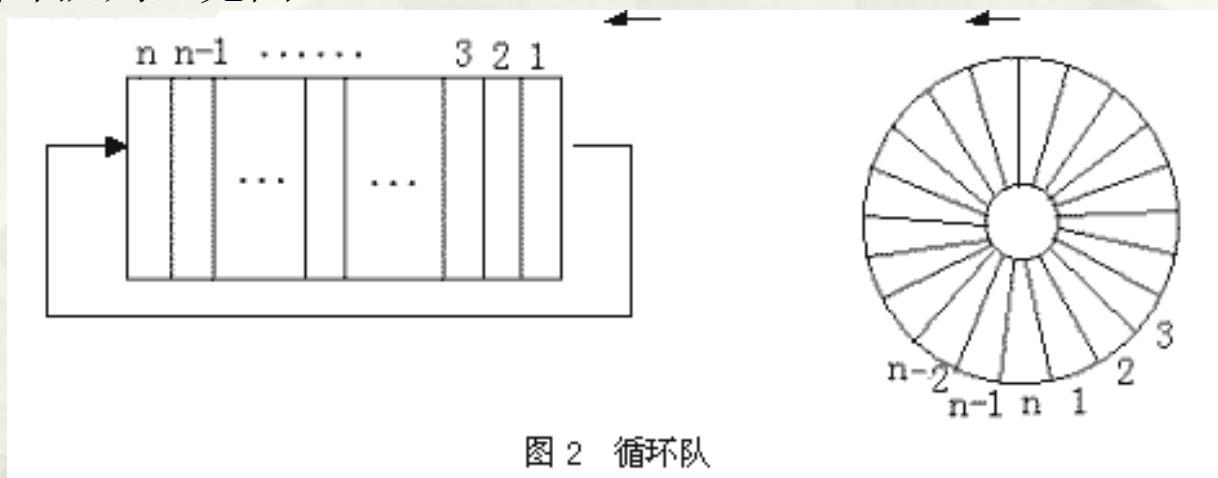
- A. 42 B. 16 C. 17 D. 41

【答案】C

【分析】元素数目可以根据公式 $(\text{tail} - \text{head} + 60) \% 60$ 得到，即 $(32 - 15 + 60) \% 60 = 17$ 。



克服假溢出的方法有两种。一种是将队列中的所有元素均向低地址区移动，显然这种方法是浪费时间的；另一种方法是将数组存储区看成是一个首尾相接的环形区域。当存放到 n 地址后，下一个地址就“翻转”为1。在结构上采用这种技巧来存储的队列称为循环队列，见图2



循环队的入队算法如下：

- 1、 $tail = tail + 1$;
- 2、若 $tail = n + 1$ ，则 $tail = 1$;
- 3、若 $head = tail$ 尾指针与头指针重合了，表示元素已装满队列，则作上溢出错误处理；
- 4、否则， $Q[tail] = x$ ，结束（ x 为新入出元素）。



A





A





A





A





A





A





A

